Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Second Assignment

Manipulator Geometry and Direct Kinematics

Author:

Allegri Luca
Laura Andrea
Zerbato Antonio

Student ID:

s7905713
s4887380
s8999827

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Simone Borelli
Luca Tarasi

December 17, 2025

Contents

Mathematical expression	Definition	MATLAB expression
$< w >$	World Coordinate Frame	w
${}^{a}_{b}R$	Rotation matrix of frame $< b >$ with respect to frame $< a >$	aRb
${}^{a}_{b}T$	Transformation matrix of frame $< b >$ with respect to frame $< a >$	aTb

Table 1: Nomenclature Table

1 Assignment description

The second assignment of Modelling and Control of Manipulators focuses on manipulators' geometry and direct kinematics.

- Download the .zip file from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling the template classes called *geometricModel* and *kinematicModel*
- Write a report motivating your answers, following the predefined format on this document.

1.1 Exercise 1

Given the following CAD model of an industrial 7 DoF manipulator:

Q1.1 Define all the model transformation matrices, by filling the structures in the *BuildTree()* function. Be careful to define the z-axis coinciding with the joint rotation axis, such that the positive rotation is the same as showed in the CAD model you received; and the x-axis, when possible, pointing to the next link. Draw on the CAD model the reference frames for each link and insert it into the report.

Q1.2 Implement the method of *geometricModel* called *updateDirectGeometry()* which computes the model transformation matrices as a function of the joint position q. Explain the method used and comment on the results obtained.

Q1.3 Implement the method of *geometricModel* called *getTransformWrtBase()* which computes the transformation matrix from the base to a given frame. Using this method, compute the following transformation matrices: ${}^{b}_{e}T$, ${}^{6}_{2}T$. Explain the method used and comment on the results obtained.

Q1.4 Run the MATLAB section Q1.4 in the given script *main.m*, which computes the configurations of the manipulators moving from an initial joint position $q_i = [\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}]^\top$ to $q_f = [\frac{5\pi}{12}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.18, \frac{\pi}{5}]^\top$. The script generates plots using methods from the given class *plotManipulators*, which must not be modified. Check that it behaves reasonably and show the obtained plots in the report.

Q1.5 Implement the method of *kinematicModel* called *getJacobianOfLinkWrtBase()* which takes as argument the link number, and returns the Jacobian of the corresponding link with respect to the base. Compute the Jacobian of **link 6** with respect to base for the final configuration q_f given in Q1.4. Explain the method used and comment on the result obtained.

Q1.6 Implement the method of *kinematicModel* called *updateJacobian()* which updates the Jacobian matrix of the manipulator for the end-effector. Compute the Jacobian for the final configuration q_f given in Q1.4. Explain the method used and comment on the results obtained.

Q1.7 Given the following joint position, $q = [0.7, -0.1, 1, -1, 0, 0.03, 1.3]^\top$ expressed in radians and meters, and the following joint velocities $\dot{q} = [0.9, 0.1, -0.2, 0.3, -0.8, 0.5, 0]^\top$ expressed in rad/s and m/s, compute the velocities of the end effector with respect to the base projected on the end effector frame, ${}^{e}v_{e/b}$ and ${}^{e}\omega_{e/b}$, using **forward kinematics**.

Remark: All the methods must be implemented for a generic serial manipulator. For instance, joint types, and the number of joints should be parameters.

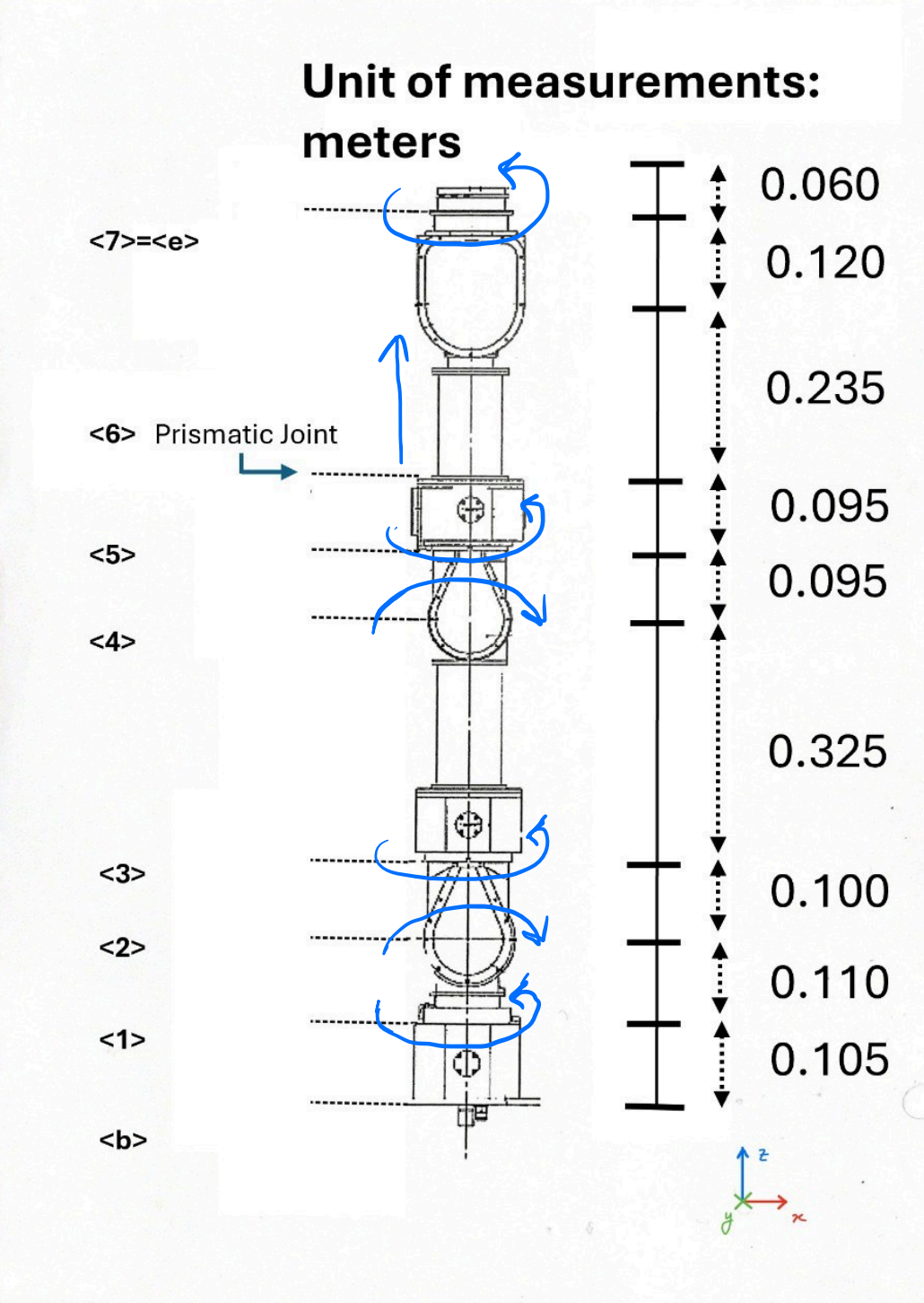

Figure 1: CAD model of the robot. The directions of motion of each joint are indicated by the blue arrows.

2 Exercise 1

2.1 Q1.1

For the first exercise we used a function called `BuildTree()` which is responsible for constructing the transformation matrices which are evaluated at zero joint configuration ($q = 0$) and describes the relative pose between two consecutive frames in the kinematic chain.

According to the provided specifications, `iTj_0` corresponds to the transformation from frame $\langle i \rangle$ to frame $\langle j \rangle$, which for $q = 0$ is equal to the transformation from frame $\langle i \rangle$ to frame $\langle i + 1 \rangle$.
In Matlab, `iTj_0` is implemented as a three-dimensional matrix:

$$iTj\_0(row, col, joint\_idx)$$

where `row` and `col` define the position within the 4×4 homogeneous matrix and the `joint_idx` identifies the joint (or link connection) in the kinematic chain.
Each slice `iTj_0(:,:,k)` therefore represents the homogeneous transformation matrix associated with the k-th joint.

For each link, the local reference frame was defined following these rules:

- the z-axis coincides with the physical joint rotation axis.
- the x-axis, when possible, points toward the next link.
- the y-axis completes a right-handed coordinate system.

All reference frames were drawn directly on the CAD model.

Each matrix `iTj_0(:,:,k)` encodes both the relative orientation and the relative position between two consecutive frames. The numerical values were extracted from the CAD model and just added in the Matlab code as we found it.

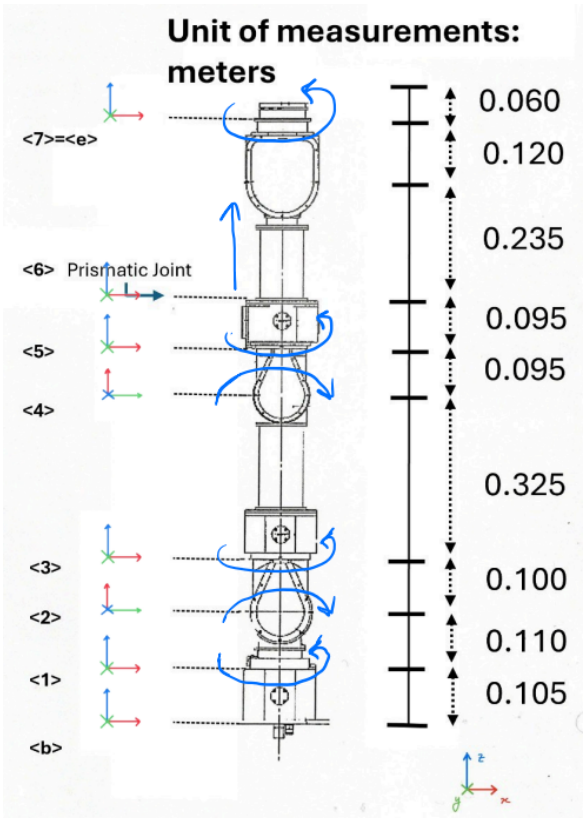

Figure 2: Build Tree Frame

The first transformation consists of an identity rotation and a translation of 0.105 m along the z-axis. This models a vertical offset between the base frame and the first joint, with no change in orientation.

$$(:,:,1) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.1050 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The same procedure was performed to compute the remaining transformation matrices:

$$(:,:,2) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0.1100 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (:,:,3) = \begin{bmatrix} 0 & 0 & 1 & 0.1000 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (:,:,4) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0.3250 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$(:,:,5) = \begin{bmatrix} 0 & 0 & 1 & 0.0950 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (:,:,6) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.0950 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (:,:,7) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.3550 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2 Q1.2

The Exercise 2 requires the implementation of the method `updateDirectGeometry()` of the geometricModel class, whose purpose is to compute the model transformation matrices as functions of the joint configuration vector q.

The class geometricModel() includes the following properties:

- `iTj_0`: set of homogeneous transformation matrices describing the geometry of the manipulator when all joint variables are zero.
- `jointType`: vector defining the type of each joint (0 for rotational, 1 for prismatic).
- `jointNumber`: total number of joints.
- `q`: joint configuration vector.
- `iTj`: transformation matrices updated according to the current joint configuration.

In the implementation of the function, a simple for loop is used to iterate over the vector $jointType$, which contains the joint variables. At each iteration of the loop, the i-th transformation matrix extracted from the initial configuration, Tij_0 , is multiplied by an additional transformation matrix that depends on the type of joint being considered.
Due to the chosen configuration structure of the local reference frame, each revolute joint performs a rotation around the local z-axis while the prismatic joints compute a translation along the z-axis. In the revolute case ,the corresponding matrix in the iTj_0 is therefore post-multiplied for this specific matrix expressing the pure rotation along the z-axis.

$$T_r = \begin{bmatrix} R_z(\theta) & 0_{3x1} \\ 0_{1x3} & 1 \end{bmatrix}$$

For the prismatic joints, the transformation matrix, expressing a translation along the local z-axis, is computed as follow:

$$T_p = \begin{bmatrix} I_{3x3} & t_z \\ 0_{1x3} & 1 \end{bmatrix}$$

2.3 Q1.3

The method `getTransformWrtBase(k)` computes the homogeneous transformation from the base frame to the k-th frame by chaining the consecutive transformations stored in `iTj`. Using this method, the transformation from the base to the end-effector frame, ${}^bT_e$, is computed by calling the function, given as input the length of the $joinType$ vector, corresponding to the total number of joints in the mechanism:

$$ {}^bT_e = \begin{bmatrix} 1.0000 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 1.1850 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

The computation of the second matrix ${}^6T_2$ was more complex. Since the transformation matrix from the base frame to the sixth frame can be expressed as:

$$ {}^0T_6 {}^6T_2 = {}^0T_2. $$

it follows that:

$$ {}^6T_2 = \left({}^0T_6\right)^{-1} {}^0T_2. $$

By calling the `getTransformWrtBase` function, we computed the matrices ${}^0T_6$ and ${}^0T_2$. The desired matrix transformation ${}^6T_2$ is obtained as:

$$ {}^6T_2 = \begin{bmatrix} 0 & 1.0000 & 0 & 0 \\ 0 & 0 & 1.0000 & 0 \\ 1.0000 & 0 & 0 & -0.6150 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} $$

2.4 Q1.4

Before examining the obtained result, let us consider the two given configurations:

- initial configuration: $q_i = [\frac{\pi}{4}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.15, \frac{\pi}{4}]^\top$
- final configuration: $q_f = [\frac{5\pi}{12}, -\frac{\pi}{4}, 0, -\frac{\pi}{4}, 0, 0.18, \frac{\pi}{5}]^\top$.

The joints intended to undergo movement are identified as:

- the first joint (revolute), with a movement of 30°
- the sixth joint (prismatic), with a translation of only 0.03 m
- the seventh joint (revolute), with a movement of 9°

The most significant movement is the 30-degree rotation of the first joint about its axis. As illustrated in the figures below, the robot transitions from the initial configuration (blue) to the final configuration (yellow), achieving the 30-degree rotation primarily through the first joint's rotation around the z axis. The movement of the sixth joint is nearly imperceptible, and, due to the absence of an attached end-effector, the movement of the last joint is also difficult to visually assess.

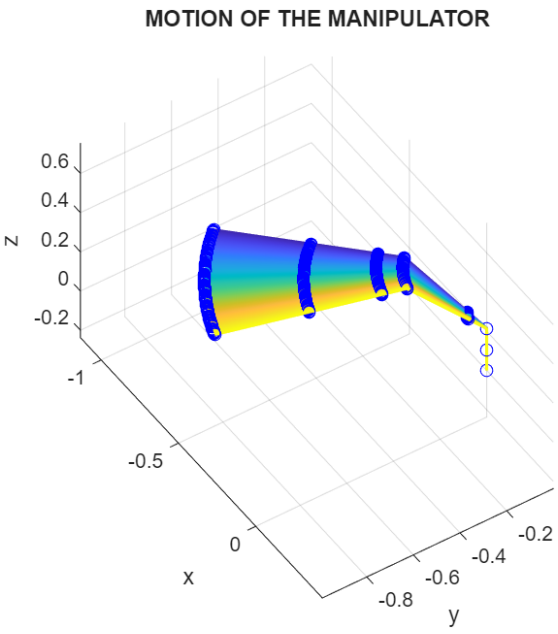

Figure 3: Final configuration of the manipulator

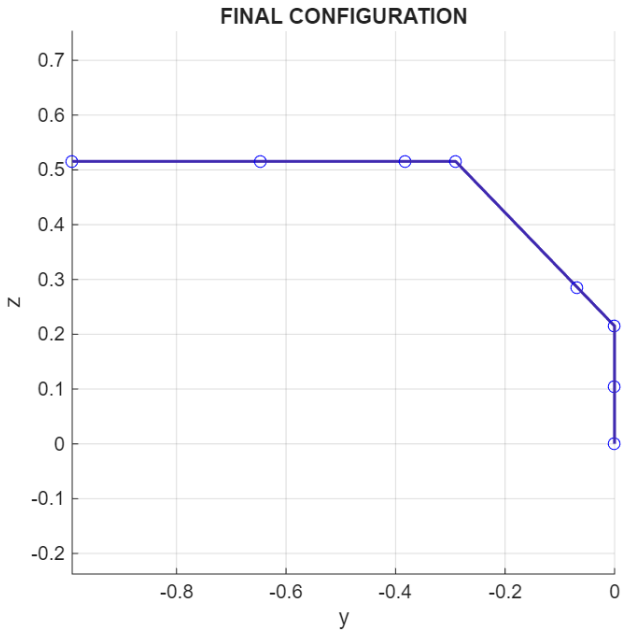

Figure 4: 3D Motion of the manipulator

2.5 Q1.5

The method *getJacobianOfLinkWrtBase(i)* takes as input the link number and computes the Jacobian $\mathbf{J_i}$ from the base to the i-th joint (the joint before the link), with respect to the base frame. This Jacobian is a $6 \times n$ matrix composed of column vectors, each of which represents the contribution of the $i-th$ joint to the velocity of the end effector (and n is the number of joints). It structured as: $\mathbf{J_i} = \begin{bmatrix} \mathbf{J_i^A} \\ \mathbf{J_i^L} \end{bmatrix}$ where $\mathbf{J_i^A}$ represents the angular velocity component and $\mathbf{J_i^L}$ represents the linear velocity component. The specific forms of these components depend on the joint type:

- Revolut joint:
  - angular component $\mathbf{J_i^A} = \mathbf{K_i}$, which is the vector of the rotation axis (aligned with the Z axis, $\mathbf{k}$, as defined in the exercise 1);
  - linear component $\mathbf{J_i^L} = \mathbf{K_i} \times \mathbf{r_{j/i}}$, the cross product between the axis of rotation and the position vector from the i-th joint to the joint I'm referring to.
- Prismatic joint:
  - angular component $\mathbf{J_i^A} = \mathbf{0}$, as a prismatic joint does not produce angular velocity;
  - linear component $\mathbf{J_i^L} = \mathbf{K_i}$, as the prismatic joint moves linearly along its $\mathbf{K}_i$ axis.

In the code implementation, the vectors $\mathbf{K_i}$ and $\mathbf{r_{j/i}}$ are retrieved as follows:

- $\mathbf{K_i}$: the first three elements of the third column of the corresponding homogeneous transformation matrix $\mathbf{T_i^0}$;
- $\mathbf{r_{j/i}}$: as the difference between the first 3 elements of the 4th column of the transformation matrix of the joint we are calculating the Jacobian, with the first 3 elements of the 4th column of the transformation matrix of the joints from the base to the i-th.

This means that each joints affects the pose of every link that comes after them. So, applying this function to Link 6 (a prismatic joint) we can say that its position and orientation depend on all the joints preceding it in the kinematic chain, but all joints that comes after doesn't influence the Jacobian Matrix. The result obtained is:

$$\mathbf{J} = \begin{bmatrix} 0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 & 0 \\ 0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 & 0 \\ 1.0000 & 0 & 0.7071 & 0 & 0 & 0 & 0 \\ 0.6477 & 0.0778 & 0.2527 & 0 & 0 & -0.2588 & 0 \\ -0.1735 & 0.2903 & -0.0677 & 0 & 0 & -0.9659 & 0 \\ 0 & 0.6705 & 0 & 0.3700 & 0 & 0 & 0 \end{bmatrix}$$

Observing the sixth-column, as expected for a prismatic joint, the angular velocity component $(\mathbf{J}_6^A)$ is zero. However, the linear velocity component $(\mathbf{J}_6^L)$ shows non-zero components along the $\mathbf{x}$ and $\mathbf{y}$ axes, indicating that the linear motion axis $(\mathbf{K}_6)$ is oriented such that its projection on the base frame has $\mathbf{x}$ and $\mathbf{y}$ components.

2.6 Q1.6

The last function, *updateJacobian*, computes the Jacobian matrix of the manipulator evaluated at the end-effector. Each column of the Jacobian corresponds to the contribution of the $i-th$ joint to the end-effector motion. The result obtained is the following:

$$\mathbf{J} = \begin{bmatrix} 0 & -0.9659 & -0.1830 & -0.9659 & -0.2588 & 0 & -0.2588 \\ 0 & 0.2588 & -0.6830 & 0.2588 & -0.9659 & 0 & -0.9659 \\ 1.0000 & 0 & 0.7071 & 0 & 0 & 0 & 0 \\ 0.9906 & 0.0778 & 0.4952 & 0 & 0 & -0.2588 & 0 \\ -0.2654 & 0.2903 & -0.1327 & 0 & 0 & -0.9659 & 0 \\ 0 & 1.0255 & 0 & 0.7250 & 0 & 0 & 0 \end{bmatrix}$$

2.7 Q1.7

The last task of the assignment aims to compute the velocities of the end effector with respect to the base projected on the end effector frame, ${}^{e}v_{e/b}$ and ${}^{e}\omega_{e/b}$, using forward kinematics.

To achieve this objective, the function `updateDirectGeometry` was first executed in order to compute the transformation matrices corresponding to the updated joint variables. Once the correct configuration of the mechanism was obtained, the Jacobian matrix was computed using the `updateJacobian` function of the kinematic model. According to the equation,

$$\begin{bmatrix} {}^{0}v_{e/b} \\ {}^{0}\omega_{e/b} \end{bmatrix} = J\dot{q}$$

the linear and angular velocities of the end effector with respect to the base frame were computed. In order to express these velocities in the end-effector frame, a change of coordinates must be applied. However, angular and linear velocities transform differently.

- The angular velocity is affected only by rotation, so the result can then be straightforwardly obtained by applying a rotational transformation
$$ {}^{e}\omega_{e/b} = {}^{0}{}_{e}R^{T\,0}\omega_{e/b} $$
- The linear velocity takes into account both rotation and translation
$$ {}^{e}\mathbf{v}_{e/b} = {}^{0}_{e}\mathbf{R}^{\top}[{}^{0}\mathbf{r}_{e}]_{\times}^{\top\,0}\boldsymbol{\omega}_{e/b} + {}^{0}_{e}\mathbf{R}^{\top\,0}\mathbf{v}_{e/b} $$
where $[{}^{0}\mathbf{r}_{e}]_{\times}^{\top}$ is the the skew-symmetric matrix associated with the position vector and $[{}^{0}\mathbf{r}_{e}]_{\times}^{\top\,0}\boldsymbol{\omega}_{e/b}$ accounts for the tangential velocity induced by the rotation of the frame.

The resulting velocities obteined with respect the end effector are:

$$ {}^{0}\omega_{e/b} = \begin{bmatrix} 0.4440 \\ -0.5585 \\ -0.5359 \end{bmatrix} \qquad {}^{0}v_{e/b} = \begin{bmatrix} -1.1688 \\ -0.7385 \\ 0.3444 \end{bmatrix} $$